

Data Structure and Algorithm

COURSE CODE: CS-201

TOPIC: DYNAMIC MEMORY ALLOCATION

COURSE INSTRUCTOR: HABIBA ARSHAD



Goals for today

To introduce :

- Dynamic Memory allocation

Memory Management with Pointers

The process of allocating and de-allocating memory is known as memory management.

Static Memory Allocation

Memory is allocated at compilation time

Dynamic Memory

Memory is allocated at running time

Static memory allocation

Memory to be allocated is specified while writing programs

```
int i , j , k ;  
char name[ 20 ];
```

Class average example

```
int main()  
{  
    int n;  
    cout << "Enter number of student" << endl;  
    cin >> n;  
    int age[n];  
}
```

Above approach has so many disadvantages

Dynamic memory allocation

Memory is allocated from Heap

Pointer is returned

Void Pointer

```
void* ptr;
```

Type casting

```
(int*) ptr; // *ptr without casting is error
```

Null pointer

Colloc

“calloc” or “contiguous allocation” method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. It is very much similar to malloc() but has two different points and these are:

It initializes each block with a default value ‘0’.

It has two parameters or arguments as compare to malloc().

Syntax:

ptr = (cast-type*)calloc(n, element-size);

here, n is the no. of elements and element-size is the size of each element.

Colloc

Calloc()

```
int* ptr = ( int* ) calloc ( 5, sizeof ( int ) );
```

4 bytes



Allocating memory dynamically

Calloc function

- Arguments
 - Number of elements
 - Size of each element

- Example

```
calloc( 10, 4) ;    // 10 ints x 4 bytes each  
calloc( 10, sizeof ( int ) ) ;  
(int*) calloc ( 1000 , sizeof ( int ) ) ;
```

Is required memory always available ?

Check if memory is allocated or not.

malloc ()

The “malloc” or “memory allocation” method in C is used to dynamically allocate a ***single large block of memory*** with the specified size.

It returns a pointer of type void which can be cast into a pointer of any form.

It doesn't initialize memory at execution time so that it has initialized each block with the default garbage value initially.

Syntax:

ptr = (cast-type*) malloc(byte-size)

For Example:

malloc ()

Malloc()

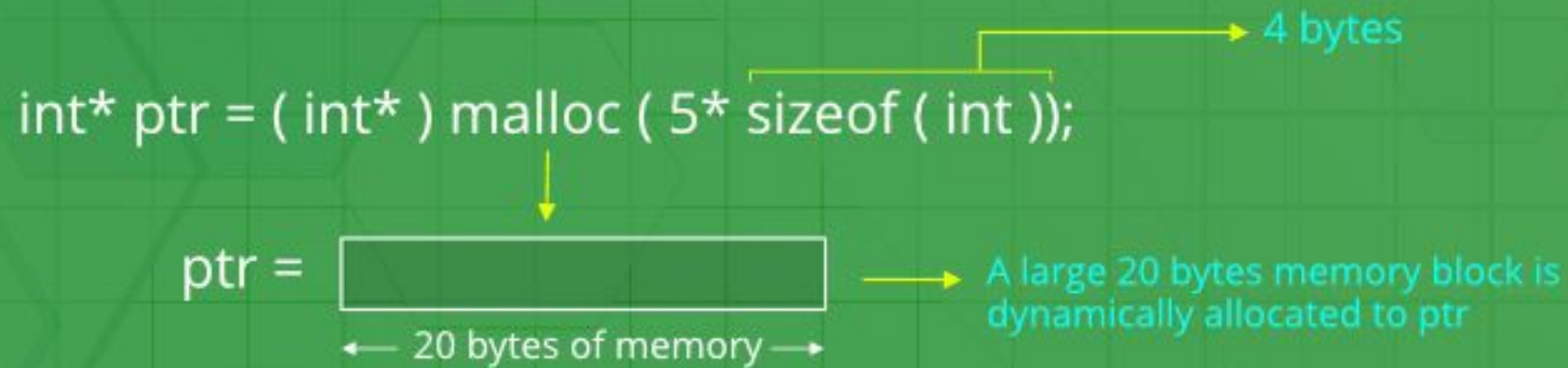
```
int* ptr = ( int* ) malloc ( 5* sizeof ( int ));
```

ptr =  

← 20 bytes of memory →

4 bytes

A large 20 bytes memory block is dynamically allocated to ptr



Increase the allocated memory (realloc)

“realloc” or “re-allocation” method in C is used to dynamically change the memory allocation of a previously allocated memory.

In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to dynamically re-allocate memory.

re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

Syntax:

ptr = realloc(ptr, newSize);

where ptr is reallocated with new size 'newSize'.

Realloc()

```
int* ptr = ( int* ) malloc ( 5* sizeof ( int ));
```

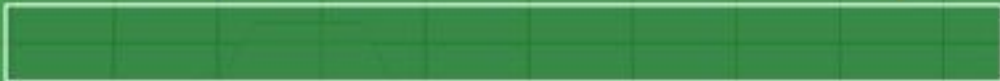
4 bytes

ptr = 

← 20 bytes of memory →

A large 20 bytes memory block is dynamically allocated to ptr

```
ptr = realloc ( ptr, 10* sizeof( int ));
```

ptr = 

← 40 bytes of memory →

The size of ptr is changed from 20 bytes to 40 bytes dynamically

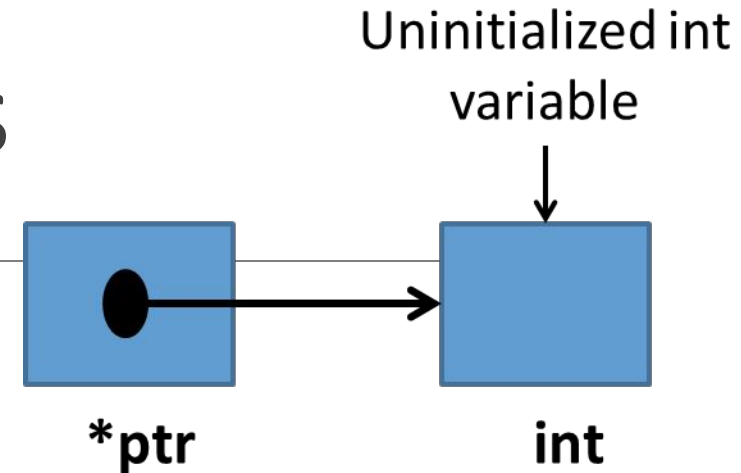
new and delete keywords

New is used to allocate memory dynamically

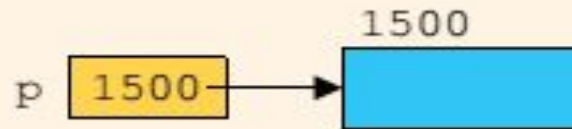
```
int*p; // declares an int pointer  
p =new int; // dynamically allocate memory  
using new  
*p = 10 // assign value to allocated memory
```

delete is used to release the allocated memory

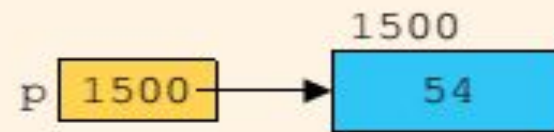
```
delete p;  
delete[] name;  
delete str;
```



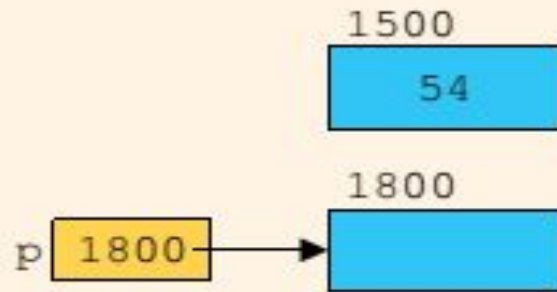
new keyword



(a) `p` after the execution of
`p = new int;`

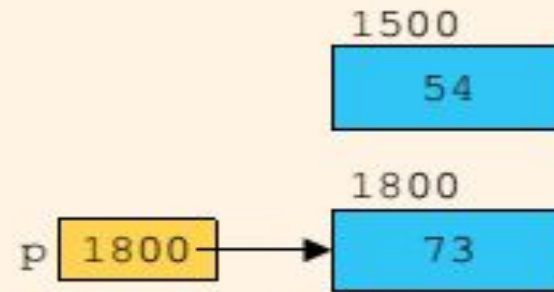


(b) `p` and `*p` after the
execution of `*p = 54;`



(c) `p` after the execution of
`p = new int;`

main



(d) `p` and `*p` after the
execution of `*p = 73;`

Problems with DMA

Memory leak

- Memory allocated but not released properly

Dangling Pointers

- Pointer pointing to illegal memory

Points to ponder

Using dynamic memory is more efficient than the static memory.

Immediately after a memory allocation call, check whether the memory has allocated successfully.

Whenever possible free the allocated memory in the same function.

Be careful about memory management to prevent memory leakage and dangling pointers.

Before exiting the program, make sure that the allocated memory has freed.

Questions?